白盒测试用例生成工具 - 实现计划

概述

一个 Python CLI 工具，使用 **AST 代码分析 + LLM 大语言模型** 自动生成白盒测试用例。

- 支持多种编程语言（Python 通过 AST 精确分析，其他语言通过 LLM 分析代码结构）
- 支持多个 LLM 后端，方便对比实验
- **闭环流程**：生成测试 → 运行测覆盖率 → 不足则反馈 LLM 补充

架构

```
                          ┌──────────────────────────────────────┐
                          │              工具内部流程              │
                          │                                      │
输入                      │  ① AST分析 → ② 构建Prompt → ③ 调用LLM  │        输出
──────────────            │       ↑                          ↓   │    ──────────────
源代码（必选）─┐           │       │    ⑤ 反馈补充     ④ 生成测试代码 │  ┌─ test_cases.json
              ├──────────→│       │    （覆盖率不足时）      ↓    │──┤
需求文档（可选）─┘         │       ←──────────────── ⑥ 运行+测覆盖率 │  ├─ test_xxx.py
                          │                                      │  └─ report.json
                          └──────────────────────────────────────┘
```

关键闭环：步骤⑥测完覆盖率后，如果未达标，把缺失的行/分支信息反馈给 LLM（步骤⑤），再生成补充测试用例，直到覆盖率达标或达到最大轮次。

项目结构

```
Assignment1/
├── main.py                          # 模块7：命令行入口
├── config.yaml                      # 配置文件（API key、模型选择）
├── requirements.txt                 # Python 依赖
├── src/
│   ├── __init__.py
│   ├── analyzers/
│   │   ├── __init__.py
│   │   └── python_analyzer.py       # 模块1：Python AST 代码分析器
│   ├── llm/
│   │   ├── __init__.py
│   │   ├── client.py                # 模块2：统一 LLM 调用客户端
│   │   └── prompts.py               # 模块3：Prompt 模板
│   ├── generators/
│   │   ├── __init__.py
│   │   └── test_generator.py        # 模块4：主流程（分析→提示→生成）
│   ├── coverage/
│   │   ├── __init__.py
│   │   └── runner.py                # 模块5：运行测试 + 覆盖率测量
│   └── experiment/
│       ├── __init__.py
│       └── tracker.py               # 模块6：实验数据记录与对比
├── output/                          # 生成的测试用例输出目录
└── targets/                         # 被测源代码存放目录
```

模块详细设计

模块1： `src/analyzers/python_analyzer.py` — AST 代码分析器

职责：解析 Python 源代码，提取白盒测试所需的代码结构信息。

提取的信息：

- 函数列表（函数名、参数、起止行号）
- 分支点（if/elif/else、for、while、try/except）
- 复合条件（and/or 表达式 → 用于条件覆盖）

- 执行路径（基于分支组合推导 → 用于路径覆盖）

核心类： `PythonAnalyzer`

- `__init__(source_code: str)` — 将代码解析为 AST
- `get_functions() -> list` — 提取函数定义
- `get_branches() -> list` — 提取所有分支点
- `get_conditions() -> list` — 提取复合布尔条件
- `analyze() -> dict` — 执行完整分析，返回结构化结果

非 Python 语言的处理：不使用 AST，而是让 LLM 充当"分析器"——把原始代码发给 LLM，让它识别代码中的分支、条件和路径。

模块2： `src/llm/client.py` — 统一 LLM 客户端

职责：提供统一接口，调用不同的 LLM API。

支持的后端（均有免费额度）：

后端	API 风格	模型
DeepSeek	OpenAI 兼容	deepseek-chat
通义千问	OpenAI 兼容	qwen-plus
硅基流动 SiliconFlow	OpenAI 兼容	各种开源模型

核心类： `LLMClient`

- `__init__(provider: str, api_key: str, model: str)` — 用配置初始化
- `chat(prompt: str) -> str` — 发送 prompt，返回响应文本

为什么用 OpenAI 兼容格式：以上三个后端都支持 OpenAI API 格式，只需要安装 `openai` 这一个包，切换 `base_url` 即可。

配置文件（`config.yaml`）：

```yaml
llm:
  deepseek:
    api_key: "sk-xxx"
    base_url: "https://api.deepseek.com"
    model: "deepseek-chat"
  qwen:
    api_key: "sk-xxx"
    base_url: "https://dashscope.aliyuncs.com/compatible-mode/v1"
    model: "qwen-plus"
  siliconflow:
    api_key: "sk-xxx"
    base_url: "https://api.siliconflow.cn/v1"
    model: "Qwen/Qwen2.5-Coder-32B-Instruct"
```

模块3： `src/llm/prompts.py` — Prompt 模板

职责：构建结构化的 prompt，引导 LLM 生成白盒测试用例。

三种 prompt：

1. **首次生成 prompt**（有 AST 分析结果时，如 Python）：

```
你是一个白盒测试专家。

## 源代码
{source_code}

## 代码结构分析（来自 AST）
- 函数列表：{functions}
- 分支点：{branches}
- 复合条件：{conditions}

## 需求文档（可选）
{requirements}

## 任务
生成测试用例以实现：
- 语句覆盖：覆盖每一行代码
- 分支覆盖：覆盖每个分支的 True/False
- 条件覆盖：覆盖每个子条件的 True/False
- 路径覆盖：覆盖所有可行的执行路径

## 输出格式
返回 JSON 数组，每个测试用例包含：
id, description, input, expected_output, covered_branches, covered_paths
```

2. **通用模式 prompt**（非 Python，无 AST，由 LLM 自行分析代码结构）：

```
你是一个白盒测试专家。

## 源代码（{language}）
{source_code}

## 任务
1. 首先分析代码结构：识别所有分支、条件和路径。
2. 然后生成覆盖所有分支/条件/路径的测试用例。
...
```

3. **补充生成 prompt**（覆盖率不足时的反馈轮次）：

```
之前生成的测试用例覆盖率不足。

## 覆盖率报告
- 当前语句覆盖率：{statement_coverage}%
- 未覆盖的代码行：{uncovered_lines}
- 未覆盖的分支：{uncovered_branches}

## 源代码（标注了未覆盖行）
{annotated_source}

## 任务
请补充测试用例，专门针对上述未覆盖的行和分支。
```

模块4： `src/generators/test_generator.py` — 主流程

职责：串联完整工作流，包含闭环反馈。

核心类： `TestGenerator`

完整工作流程：

```
1. 读取源代码文件
2. 检测编程语言（根据文件扩展名）
3. 如果是 Python → 运行 PythonAnalyzer（AST 精确分析）
   如果是其他语言 → 使用 LLM 通用模式（LLM 自行分析代码结构）
4. 构建 prompt（代码 + 分析结果 + 可选的需求文档）
5. 调用 LLM，生成初始测试用例
6. 解析 LLM 响应（从文本中提取 JSON 测试用例）
7. 生成可运行的测试文件（Python → pytest，JS → jest）
8. 调用 CoverageRunner 运行测试并测量覆盖率      ← 新增
9. 如果覆盖率不足且未达最大轮次：                  ← 新增（闭环）
   - 构建"补充 prompt"（包含未覆盖行信息）
   - 再次调用 LLM 生成补充测试
   - 合并测试用例，回到步骤 8
10. 保存最终输出到 output/ 目录
```

方法：

- `__init__(source_path, requirement_path=None, llm_provider="deepseek", max_rounds=3)`
- `run() -> dict` — 执行完整流程（含闭环），返回结果

- `_detect_language(path) -> str` — 检测语言
- `_parse_llm_response(response) -> list` — 从 LLM 文本中提取 JSON
- `_generate_test_file(test_cases, language) -> str` — 生成可运行的测试代码
- `_save_outputs(test_cases, test_code, coverage_report)` — 写入 output/ 目录

模块5： `src/coverage/runner.py` — 测试运行 + 覆盖率测量（新增）

职责：运行生成的测试代码，测量覆盖率，返回结构化的覆盖率报告。

支持的语言：

语言	测试框架	覆盖率工具
Python	pytest	coverage.py
JavaScript	jest / mocha	nyc (istanbul)

核心类： `CoverageRunner`

- `__init__(source_path, test_path, language)`
- `run() -> dict` — 运行测试并返回覆盖率报告
- `_run_python_coverage()` — 用 `coverage run -m pytest + coverage json`
- `_run_js_coverage()` — 用 `npx jest --coverage`
- `get_uncovered_lines() -> list` — 获取未覆盖的行号
- `get_uncovered_branches() -> list` — 获取未覆盖的分支

返回的覆盖率报告：

```
{
  "statement_coverage": 85.5,
  "branch_coverage": 72.0,
  "uncovered_lines": [45, 67, 89],
  "uncovered_branches": ["L45: if x > 1000000000 → True"],
  "test_results": {
    "total": 15,
    "passed": 14,
    "failed": 1
  }
}
```

模块6： `src/experiment/tracker.py` — 实验数据记录

职责：记录每次运行的数据，支持不同 LLM 后端的对比。

记录的数据（每次运行）：

- 使用的 LLM 后端和模型名
- 分析的源文件
- 生成的测试用例数量
- 耗时（API 调用时间）
- 闭环轮次数
- 最终覆盖率（语句/分支）
- 测试通过率

核心类： `ExperimentTracker`

- `record(run_data: dict)` — 保存一次运行的数据
- `compare() -> str` — 生成多次运行的对比表格
- `save_report(path)` — 导出为 JSON/Markdown

对比表格示例：

LLM	用例数量	生成耗时	语句覆盖	分支覆盖	闭环轮次
DeepSeek-V3	24	8.2s	95.2%	88.0%	2
Qwen-Plus	20	6.5s	87.3%	76.0%	3
SiliconFlow	18	9.1s	82.1%	72.0%	3

模块7： `main.py` — 命令行入口

使用方式：

```python
# 基本用法：用 DeepSeek 分析 Python 文件（含闭环补充）
python main.py --source targets/convert_number_to_words.py --llm deepseek

# 附带需求文档（辅助 LLM 理解代码意图）
python main.py --source targets/index.js --llm qwen --requirement docs/spec.txt

# 对比多个 LLM 的生成效果
python main.py --source targets/convert_number_to_words.py --compare

# 只生成测试用例，不运行覆盖率闭环
python main.py --source targets/code.py --llm deepseek --no-coverage

# 设置最大闭环轮次
python main.py --source targets/code.py --llm deepseek --max-rounds 5
```

参数说明:

- `--source` （必选）：被测源代码文件路径
- `--llm` ： 使用的 LLM 后端（deepseek / qwen / siliconflow）
- `--requirement` ： 可选的需求文档，为 LLM 提供上下文
- `--compare` ： 用所有已配置的 LLM 运行并输出对比表格
- `--output` ： 输出目录（默认 ./output）
- `--no-coverage` ： 只生成测试用例，不运行覆盖率闭环
- `--max-rounds` ： 最大闭环轮次（默认 3）

实现顺序

步骤	模块	难度	依赖
1	`python_analyzer.py` （AST 分析器）	简单	无
2	`client.py` （LLM 客户端）	简单	`openai` 包
3	`prompts.py` （Prompt 模板）	中等	模块1的输出格式
4	`test_generator.py` （主流程）	中等	模块1、2、3
5	`runner.py` （覆盖率测量）	中等	`coverage` 、 `pytest`
6	闭环集成（在 test_generator 中加入反馈逻辑）	中等	模块4、5

步骤	模块	难度	依赖
7	`tracker.py` （实验记录）	简单	模块4输出格式
8	`main.py` （CLI 入口）	简单	所有模块

依赖（`requirements.txt`）

```
openai>=1.0.0      # 调用 LLM API（所有后端都兼容 OpenAI 格式）
pyyaml>=6.0        # 读取 config.yaml
coverage>=7.0      # Python 覆盖率测量
pytest>=7.0        # 运行 Python 测试
```

输出样例

test_cases.json（结构化测试用例）


```json
{
  "source_file": "convert_number_to_words.py",
  "language": "python",
  "llm_provider": "deepseek",
  "llm_model": "deepseek-chat",
  "rounds": 2,
  "analysis": {
    "functions": ["..."],
    "branches": ["..."],
    "conditions": ["..."]
  },
  "test_cases": [
    {
      "id": "TC001",
      "round": 1,
      "description": "测试负数输入，覆盖 if num < 0 → True 分支",
      "input": {"num": -5},
      "expected_output": "minus five",
      "covered_branches": ["L12: if num < 0 → True"],
      "covered_paths": ["path1: L10→L12→L13→L15"]
    }
  ],
  "coverage": {
    "statement_coverage": 95.2,
    "branch_coverage": 88.0,
    "uncovered_lines": [102, 105]
  }
}
```

test_convert_number_to_words.py（可运行的测试代码）

```python
import pytest
from convert_number_to_words import num_to_words

class TestNumToWords:
    def test_negative_number(self):
        """TC001: 覆盖分支 L12 if num < 0 → True"""
        assert num_to_words(-5) == "minus five"

    def test_zero(self):
        """TC002: 覆盖分支 L14 if num == 0 → True"""
        assert num_to_words(0) == "zero"

    # --- 第2轮补充（覆盖率反馈后生成）---
    def test_billion(self):
        """TC015: 覆盖 L102 处理十亿的逻辑"""
        assert num_to_words(1000000000) == "one billion"
```